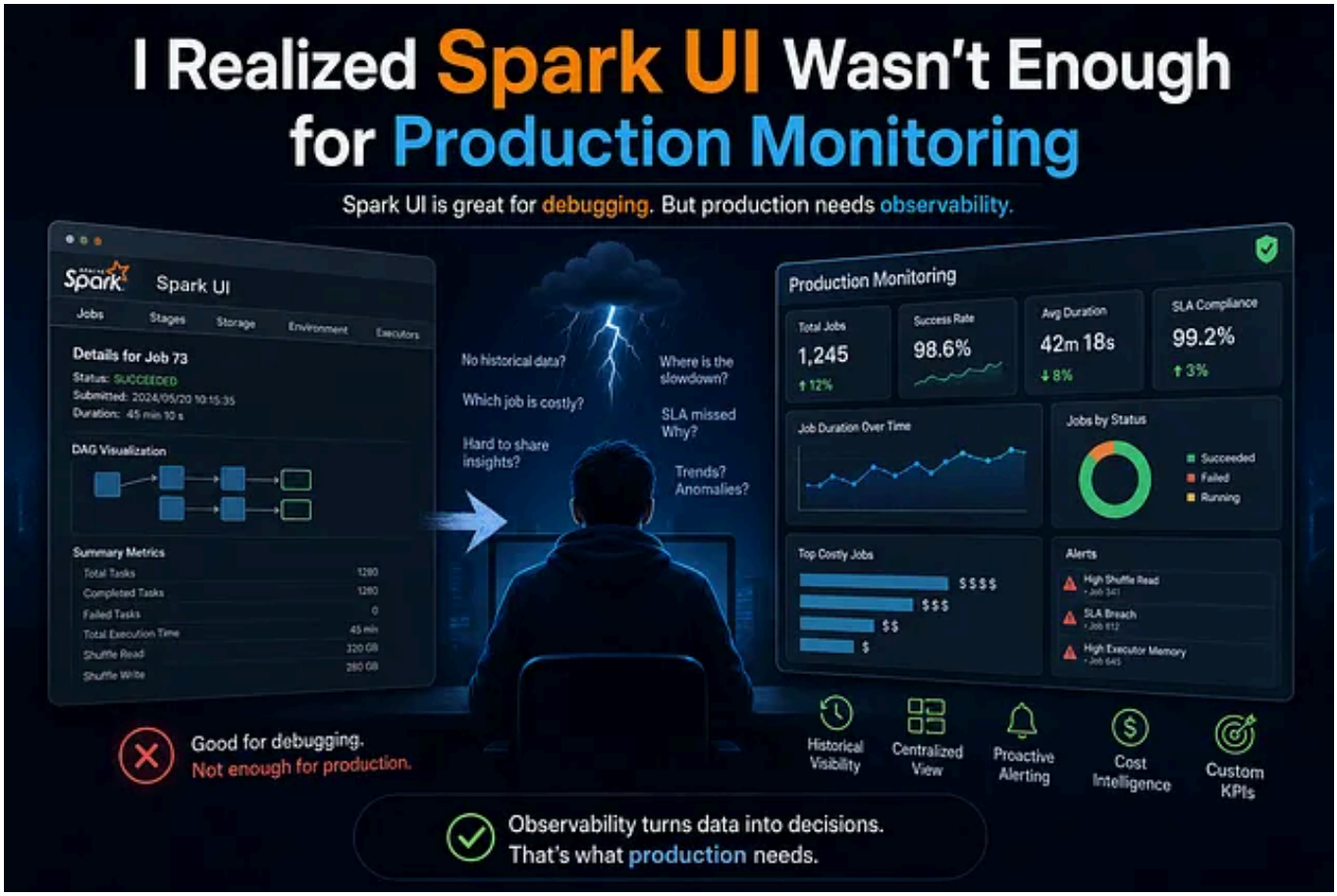

I Realized Spark UI Wasn’t Enough for Production Monitoring

Anurag Sharma 6 min read · Jun 15, 2026

Apache Spark is one of the most powerful technologies in modern data engineering. It can process terabytes of data, power massive ETL pipelines, drive real-time analytics, and scale distributed workloads across entire organizations. But after working with Spark in production environments, I realized something important:

> **Building Spark pipelines is easy compared to monitoring them in production.**

At a small scale, Spark’s built-in UI feels incredibly useful. You can inspect stages, analyze DAGs, check executor usage, and debug failed jobs quickly. But once your workloads grow across multiple teams, clusters, and business-critical pipelines, the Spark UI starts showing its limits. And that’s exactly where the real monitoring problems begin.

The Moment I Realized the Problem

Everything looked normal.

A nightly ETL pipeline that usually finished in 45 minutes suddenly started taking more than 3 hours.

Downstream dashboards failed.
Business reports were delayed.
Alerts started firing from other systems.

The first instinct was obvious:
Open the Spark UI.

But that’s where the frustration started.

We could see the current execution.
We could inspect stages.
We could analyze shuffle metrics.

But we couldn’t answer the questions that actually mattered:

- Was this slowdown gradual or sudden?
- Which release introduced the degradation?
- Was this happening across other jobs, too?
- How much extra infrastructure cost did this create?
- Was data skew increasing over time?
- Did executor memory usage trend upward over the past month?
- Which pipelines were most inefficient historically?

The Spark UI had metrics.

But it didn’t have observability.

That was the moment I realized:

> **Spark UI is excellent for debugging jobs.**
> **It is not designed for production-grade monitoring.**

Spark UI Is Great — But Only for a Specific Purpose

To be fair, Spark UI does its original job very well.

It was built primarily for:

✅ Debugging individual applications
✅ Inspecting stages and tasks
✅ Understanding execution plans
✅ Identifying failed jobs
✅ Viewing executor-level metrics

And for development environments, it works beautifully.

But production systems demand something completely different.

Modern data platforms need:

- Historical visibility
- Cross-job monitoring
- Business-aware metrics
- Cost analytics
- Centralized dashboards
- Intelligent alerting
- Trend analysis
- Team collaboration
- SLA tracking
- Observability at scale

And this is where the native Spark UI begins to fall short.

1. Yesterday’s Metrics Are Gone

One of the first pain points teams experience is the lack of historical visibility.

Spark UI metrics are transient.

Once applications expire from the retention window, the execution details disappear.

That means teams lose the ability to:

- Compare job performance across weeks
- Analyze long-term degradation
- Understand release impacts
- Identify seasonal trends
- Track infrastructure optimization improvements

This becomes a serious operational problem in production environments.

Imagine trying to answer:

> “Why did this pipeline become 40% slower over the last two months?”

Without historical metrics, teams are forced to rely on logs, screenshots, or manual exports.

That’s not observability.

That’s operational guesswork.

2. No Single View Across the Spark Ecosystem

In real-world enterprises, Spark workloads are rarely isolated.

You typically have:

- Multiple clusters
- Hundreds of jobs
- Different environments
- Multiple teams
- Streaming + batch workloads
- Shared infrastructure

But Spark UI operates application-by-application.

There is no centralized visibility layer.

Engineers constantly jump between:

- Application UIs
- Cluster dashboards
- Driver logs
- Executor logs
- Cloud monitoring systems

This context switching creates operational chaos.

And during incidents, every minute matters.

Modern monitoring systems require a unified view of the entire Spark ecosystem — not fragmented interfaces spread across clusters.

3. System Metrics Don’t Explain Business Impact

Spark provides excellent low-level metrics:

- Shuffle read/write
- Task duration
- Executor memory
- CPU usage
- Storage metrics

But business stakeholders rarely care about executor memory consumption.

They care about:

- SLA compliance
- Data freshness
- Pipeline completion times
- Revenue-impacting failures
- Delayed dashboards
- Customer-facing latency

This creates a major gap between technical monitoring and business observability.

For example:

A Spark job may complete successfully from a technical perspective while still violating a critical business SLA.

Spark UI has no concept of business metrics.

It understands executors.

Not business outcomes.

4. Monitoring Is Reactive Instead of Proactive

One of the biggest weaknesses of traditional Spark monitoring is that it’s mostly reactive.

Teams investigate problems after failures happen.

But production-grade platforms need proactive intelligence.

Modern systems should detect:

- Runtime anomalies
- Resource inefficiencies
- Gradual degradation
- Data skew trends
- Executor instability
- SLA risk patterns

Before failures occur.

Spark UI offers very limited alerting capabilities.

There is no built-in anomaly detection.
No predictive insights.
No intelligent thresholding.

And as workloads scale, reactive monitoring becomes an extremely expensive operation.

5. Long-Term Trend Analysis Is Painful

Distributed systems rarely fail instantly.

Most production problems emerge slowly.

Examples:

- Increasing shuffle size
- Gradual memory pressure

- Growing executor skew
- Rising infrastructure costs
- Slower joins after schema growth

These patterns only become visible through long-term trend analysis.

But Spark UI was never designed for this.

Its data model is optimized for immediate execution visibility — not historical analytics.

As a result, teams often end up building manual workflows involving:

- Log exports
- External databases
- Grafana dashboards
- Custom scripts
- Ad-hoc analytics pipelines

Ironically, engineers start building data pipelines just to monitor their data pipelines.

6. Cross-Team Collaboration Becomes Difficult

Modern data platforms are collaborative ecosystems.

Data engineers, analysts, platform teams, DevOps engineers, and business stakeholders all need visibility into workloads.

But Spark UI isn’t designed for easy sharing.

Common problems include:

- Direct cluster access requirements
- Limited reporting capabilities
- No business-friendly dashboards
- Poor governance controls
- Difficult metric sharing

This creates dependency bottlenecks where only a few engineers truly understand platform behavior.

Observability should democratize visibility.

7. There Is Almost No Cost Intelligence

This becomes especially painful in cloud environments.

Organizations spend thousands — sometimes millions — on Spark infrastructure.

Yet many teams still cannot answer basic questions like:

- Which pipeline costs the most?
- Which workloads are inefficient?
- Which jobs waste executor resources?

- Which transformations increase cloud spending?

Spark UI provides infrastructure metrics.

But it provides almost no financial visibility.

And without cost observability, optimization becomes nearly impossible.

This is one of the most overlooked problems in modern data engineering.

8. Custom KPIs Are Missing Entirely

Every organization has unique operational requirements.

Some teams care about:

- Records processed per second
- Data quality scores
- Domain-specific SLA metrics
- Streaming lag
- Error percentages
- Pipeline efficiency ratios

These metrics are critical to business operations.

But Spark UI only supports predefined system metrics.

There’s no native way to define organization-specific KPIs that matter to stakeholders.

And this becomes a huge limitation at scale.

The Realization: Spark Monitoring Is a Platform Engineering Problem

Over time, I realized something important:

> Spark monitoring is no longer just a debugging problem.
> It’s a platform engineering challenge.

Modern data organizations need observability platforms — not just execution UIs.

That means building systems capable of:

- ✅ Historical metric storage
- ✅ Real-time alerting
- ✅ Centralized visibility
- ✅ Cross-job analytics
- ✅ Cost tracking
- ✅ Custom KPIs
- ✅ Business-aware monitoring
- ✅ Team-wide dashboards
- ✅ Long-term trend analysis
- ✅ Proactive anomaly detection

At this point, monitoring becomes an architecture problem.

Not a UI problem.

What Modern Spark Monitoring Actually Looks Like

A production-grade Spark observability architecture often looks something like this:

```
Spark Jobs
    ↓
Custom Spark Listener
    ↓
Kafka / PubSub / Event Streaming
    ↓
Metrics Storage Layer
(PostgreSQL / Elasticsearch / BigQuery / Delta Lake)
    ↓
Dashboard & Visualization Layer
(Grafana / Custom UI / Datadog)
    ↓
Alerting + Trend Analysis + SLA Monitoring
```

This is the direction many mature data engineering organizations eventually move toward.

Because at scale:

> Observability becomes just as important as computation.

The Hidden Truth About Modern Data Platforms

Ironically, many organizations invest heavily in scalable compute infrastructure while having very little visibility into how their pipelines actually behave in production.

They can process petabytes of data.

But they struggle to answer:

- Why are jobs slowing down
- Which pipelines are becoming unstable
- Where costs are exploding
- Which workloads violate SLAs
- How infrastructure changes affect performance

This is the observability gap in modern data engineering and it’s growing rapidly.

Final Thoughts

Spark changed the way organizations process data. But as systems become larger and more distributed, monitoring can no longer remain an afterthought.

The future of Spark isn’t only about faster computation.

It’s about:

- Better observability
- Smarter monitoring
- Business-aware insights
- Historical intelligence

- Platform-wide visibility

Because the best data platforms are not simply the ones processing the most data.

They are the ones teams can actually understand, trust, optimize, and operate confidently in production.

And that realization completely changed the way I think about building Spark systems.

Data Engineering | Big Data | Apache Spark | Data Engineer | Automation

Written by Anurag Sharma

125 followers · 5 following

Data Engineering Specialist with 10+ exp. Passionate about optimizing pipelines, data lineage, and Spark performance and sharing insights to empower data pros!

Edit profile